Counterexample Analysis Report

Project: University Course Management System
Tool Used: Alloy Analyzer
Verification Phase: Structural Verification and Counterexample Analysis

1. Introduction

This report explains the counterexample analysis performed on the University Course Management System using Alloy Analyzer. The purpose of this verification activity was to evaluate whether the system constraints and relational models were logically correct and capable of preventing invalid states.

The Alloy model was developed using signatures, relations, predicates, and facts. Students, courses, and enrollments were represented as relational entities inside the model. Constraints were then applied to verify enrollment capacity, duplicate enrollments, and student course limits.

The verification process helped identify whether the model could detect inconsistencies automatically. To test this capability, an intentionally conflicting constraint was introduced into the system. This allowed Alloy Analyzer to generate a counterexample and demonstrate how incorrect specifications can lead to inconsistent states.

2. Relational Model Verification

The structural model of the University Course Management System consisted of three main signatures: Student, Course, and Enrollment. The Enrollment relation connected students with courses. Additional constraints were added to ensure system correctness.

The following important constraints were verified:

1. A course cannot exceed its maximum capacity.
2. A student cannot enroll in the same course more than once.
3. A student cannot register in more than five courses.

These constraints were implemented using Alloy facts and relational expressions. The model was executed using the Alloy Analyzer execution engine to validate whether the specifications remained consistent under different generated instances.

3. Counterexample Analysis

To perform counterexample analysis, a deliberately incorrect constraint was added into the Alloy model. The incorrect constraint stated that the number of enrollments in a course must always be greater than the course capacity.

This constraint directly conflicted with the valid system rule that enrollment count must remain less than or equal to course capacity. Because of this contradiction, Alloy Analyzer failed to generate a valid instance for the model.

The incorrect constraint used was:

#(course.c) > c.capacity

After executing the model, Alloy Analyzer reported inconsistency in the specification. The tool was unable to produce a satisfiable model because both constraints could not remain true at the same time.

This result successfully demonstrated the ability of Alloy Analyzer to identify invalid system states automatically. The counterexample proved that structural verification was functioning correctly and that contradictory rules could be detected during formal analysis.

4. Results and Findings

The verification process confirmed that the valid UCMS model remained structurally correct when proper constraints were applied. The generated visual instances showed valid relationships between students, courses, and enrollments.

The intentionally introduced contradiction produced an inconsistent state, which Alloy Analyzer successfully detected. This verified that the model constraints were meaningful and capable of preventing logically invalid system configurations.

The experiment also highlighted the importance of formal verification in software systems. Without structural verification, contradictory requirements may remain undetected and later cause system failures during implementation.

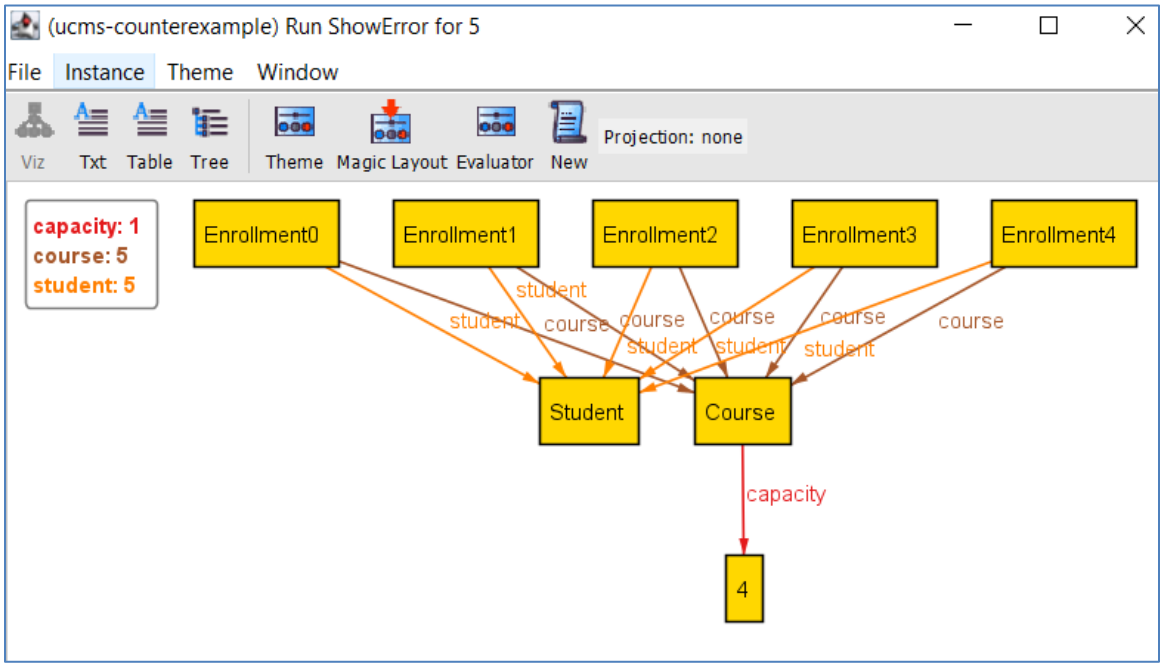
5. Conclusion

The counterexample analysis demonstrated that Alloy Analyzer is an effective tool for structural verification of software systems. The University Course Management System model successfully verified relational consistency, enrollment rules, and system constraints.

The generated counterexample proved that the Alloy model could detect inconsistencies and invalid states automatically. This process improved confidence in the correctness of the system specification and supported the overall goals of Software Verification and Validation.

The verification results obtained from Alloy Analyzer will also support future stages of the SVV pipeline, including validation and final system documentation.

Screenshots:



```
module counterexample

sig Student {}

sig Course {
    capacity : one Int
}

sig Enrollment {
    student : one Student,
    course : one Course
}

fact WrongConstraint {
    all c : Course |
    #(course.c) > c.capacity
}

pred ShowError {}

run ShowError for 5
```